Dollor.ai Platform - Complete Technical Report

Version: 1.0 **Date:** December 9, 2025 **Platform:** iOS Customer App + Python Backend + AWS Infrastructure

---

Table of Contents

---

1. Executive Summary

Dollor.ai is a food delivery platform offering $1 delivery fees. The platform consists of:

- **3 iOS Apps**: Customer, Restaurant, Delivery Driver - **Python Backend**: FastAPI with SQLAlchemy ORM - **Database**: PostgreSQL on AWS RDS - **Infrastructure**: AWS App Runner (auto-scaling 1-25 instances) - **Payments**: Stripe (Payment Intents, Connect, Webhooks) - **Real-time**: Firebase Firestore (fallback/real-time features)

Key Metrics

- **14+ Database Tables** with full relational integrity - **50+ API Endpoints** across all services - **5 AI Employees** automating platform operations - **25+ iOS Views** with

comprehensive data binding

---

2. Google Cloud API Security

2.1 Current Configuration

Client ID: `107524350806-ign58n65jrc4i0ab8audp3qgp24b37if.apps.googleusercontent.com`

2.2 Security Steps to Implement

Step 1: Access Google Cloud Console

```
https://console.cloud.google.com/apis/credentials
```

Step 2: Configure OAuth Consent Screen

1. Navigate to **APIs & Services > OAuth consent screen** 2. Set App name: `Dollor.ai` 3. Add authorized domains: `dollor.ai` 4. Add support email and developer contact

Step 3: Restrict OAuth Client ID

1. Go to **APIs & Services > Credentials** 2. Click on your iOS OAuth Client ID 3. Under **Application type**, ensure "iOS" is selected 4. Add **Bundle ID restriction**: `com.dollor.customer`

Step 4: Enable Required APIs Only

```
 - Google Sign-In API (enabled)
- Maps SDK for iOS (enabled)
- Places API (enabled)
- Directions API (enabled)
- Geocoding API (enabled)
```

Step 5: Set API Key Restrictions

1. Create separate API keys for: - iOS app (restrict to bundle ID) - Backend server (restrict to IP addresses) 2. Apply API restrictions (only enable needed APIs)

Step 6: Implement Key Rotation Schedule

- Rotate API keys every 90 days - Use environment variables, never hardcode - Monitor usage in Cloud Console

2.3 Security Checklist

Item	Status	Action
Bundle ID restriction	Required	Add `com.dollor.customer`
API restrictions	Required	Limit to Maps, Places, Directions
Referrer restrictions	Required	Add `dollor.ai` domain
Billing alerts	Recommended	Set at $100, $500, $1000
Usage quotas	Recommended	Set daily limits

---

3. iOS App Architecture

3.1 Technology Stack

Component	Technology
UI Framework	SwiftUI
State Management	@Published, @StateObject, @EnvironmentObject

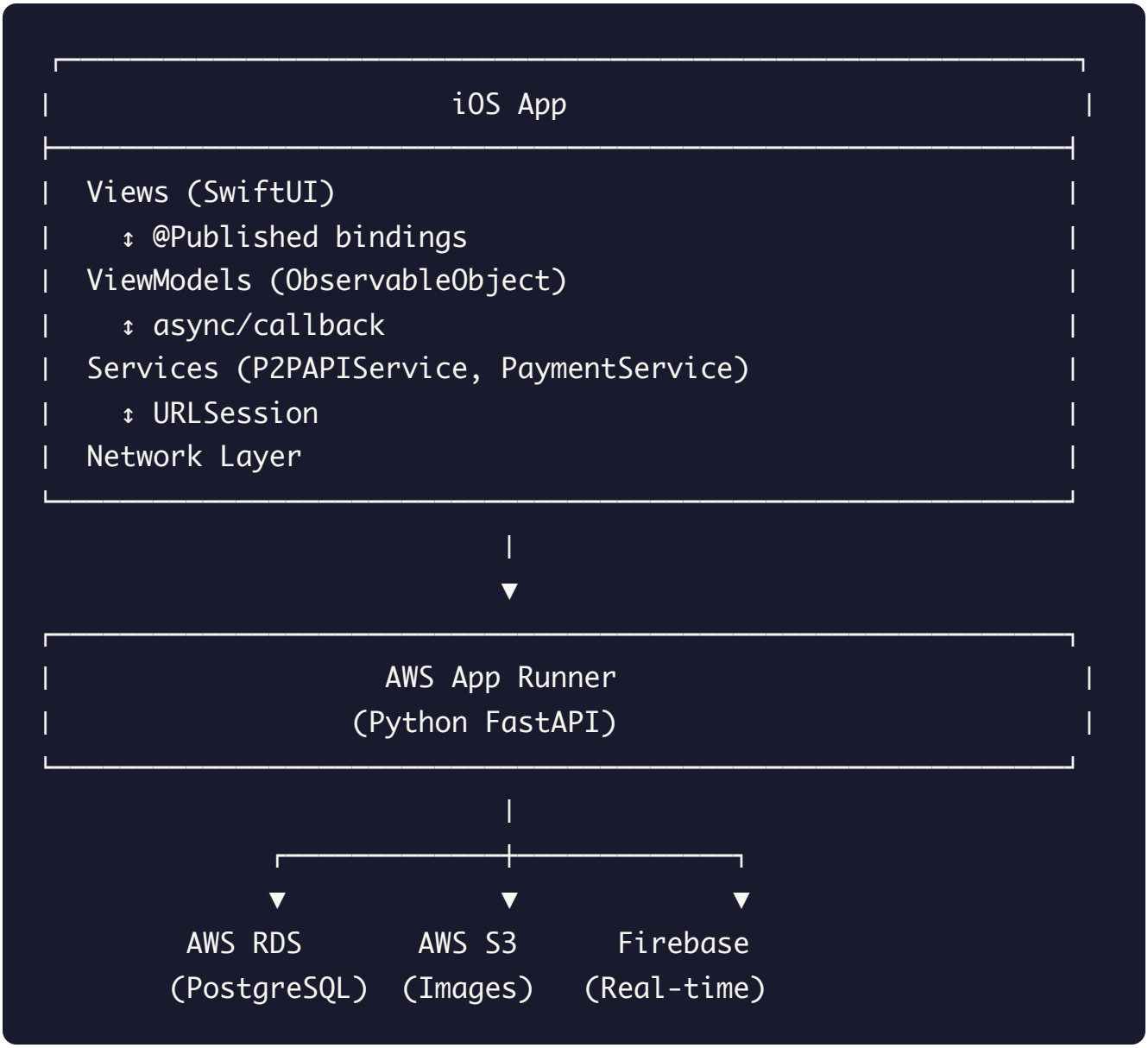
Local Storage	@AppStorage (UserDefaults)
Networking	URLSession + Codable
Authentication	Google Sign-In SDK, Apple Sign-In
Maps	Google Maps SDK for iOS
Payments	Stripe iOS SDK
Real-time	Firebase Firestore

3.2 Project Structure

```
eatfaircustomer/
├── App/
|   └── eatfaircustomerApp.swift
├── Models/
|   ├── MenuItem.swift
|   ├── Order.swift
|   ├── Restaurant.swift
|   └── CartItem.swift
├── ViewModels/
|   ├── AuthViewModel.swift
|   ├── HomeViewModel.swift
|   ├── CartViewModel.swift
|   ├── MenuViewModel.swift
|   ├── OrderHistoryViewModel.swift
|   ├── AddressViewModel.swift
|   └── MultiRestaurantCartViewModel.swift
├── Views/
|   ├── LoginView.swift
|   ├── HomeView.swift
|   ├── RestaurantDetailView.swift
|   ├── CheckoutView.swift
|   ├── OrderTrackingView.swift
|   └── [20+ additional views]
```

```
├── Services/
|   ├── PaymentService.swift
|   ├── ACHPaymentService.swift
|   └── LocationManager.swift
└── Shared/ (EatFairShared package)
    ├── P2PAPIService.swift
    ├── AppConfig.swift
    └── Models/
```

3.3 Data Flow Architecture



4. View-by-View Data Flow Analysis

4.1 Authentication Views

LoginView.swift

Aspect	Details
Data Source	P2P Backend (`/api/customer/login`)
ViewModel	AuthViewModel
Local Storage	JWT token in Keychain, user info in @AppStorage
Firebase Usage	None
Fallback	None - authentication requires backend

Data Flow:

```
User Input → AuthViewModel.login() → P2PAPIService.customerLogin()
    → Backend validates → Returns JWT + user info
    → Stored in Keychain + @AppStorage
    → isAuthenticated = true → Navigate to HomeView
```

SignUpView.swift

Aspect	Details
Data Source	P2P Backend (`/api/customer/register`)
ViewModel	AuthViewModel
Validation	Client-side + server-side

Firebase Usage	None

4.2 Home & Discovery Views

HomeView.swift

Aspect	Details
Data Source	P2P Backend (`/api/restaurants`)
ViewModel	HomeViewModel
Local Storage	Recent searches in @AppStorage
Firebase Usage	Fallback for restaurant list
Caching	In-memory restaurant cache

Data Flow:

```
HomeView appears → HomeViewModel.fetchRestaurants()
    → P2PAPIService.fetchRestaurants()
    → Success: Display restaurant grid
    → Failure: Try Firebase fallback
        → FirebaseService.fetchRestaurants()
        → Display cached/Firebase data
```

Displayed Data: - Featured restaurants (curated list) - Nearby restaurants (location-based) - Active order status (if any) - Promotional banners

RestaurantDetailView.swift

Aspect	Details
Data Source	P2P Backend (`/api/restaurants/{id}`)

ViewModel	MenuViewModel
Firebase Usage	Fallback for menu items
Real-time	None

Data Flow:

```
Restaurant tapped → MenuViewModel.fetchMenu(restaurantId)
    → P2PAPIService.fetchRestaurantDetail()
    → Parse menu categories and items
    → Display grouped by category
```

4.3 Cart & Checkout Views

CartView.swift / MultiRestaurantCartView.swift

Aspect	Details
Data Source	Local (CartViewModel)
ViewModel	CartViewModel / MultiRestaurantCartViewModel
Local Storage	Cart persisted in @AppStorage
Firebase Usage	None
Sync	Cart synced to backend on checkout

Data Flow:

```
Add to Cart → CartViewModel.addItem()
    → Update local cart array
    → Persist to @AppStorage
    → UI updates via @Published
```

```
Modify Quantity → CartViewModel.updateQuantity()
    → Recalculate totals
    → Update @AppStorage
```

CheckoutView.swift / MultiRestaurantCheckoutView.swift

Aspect	Details
Data Source	P2P Backend (multiple endpoints)
ViewModel	CartViewModel
Payment	Stripe SDK + P2P Backend
Promo Codes	Backend validation only

Data Flow:

```
Checkout initiated:
1. Validate delivery address → P2PAPIService.validateAddress()
2. Apply promo code (if any) → P2PAPIService.validatePromoCode()
3. Create payment intent → P2PAPIService.createPaymentIntent()
4. Stripe SDK processes payment
5. Confirm order → P2PAPIService.createOrder()
6. Clear cart → CartViewModel.clearCart()
7. Navigate to OrderSuccessView
```

4.4 Order Management Views

OrderHistoryView.swift

Aspect	Details

Data Source	P2P Backend (`/api/customer/orders`)
ViewModel	OrderHistoryViewModel
Firebase Usage	None
Refresh	Pull-to-refresh + 30s auto-refresh

OrderTrackingView.swift / DeliveryTrackingView.swift

Aspect	Details
Data Source	P2P Backend + Firebase (real-time)
ViewModel	OrderTrackingViewModel
Firebase Usage	Real-time driver location updates
Maps	Google Maps SDK

Data Flow:

```
Order placed → OrderTrackingView
    → Fetch order status from P2P
    → Subscribe to Firebase for real-time updates:
        - Driver location (lat/lng)
        - Order status changes
        - ETA updates
    → Update map markers in real-time
```

4.5 Profile & Settings Views

ProfileView.swift

Aspect	Details

Data Source	P2P Backend (`/api/customer/profile`)
ViewModel	AuthViewModel
Local Storage	Profile cached in @AppStorage
Edit	Updates sent to backend

AddressManagementView.swift

Aspect	Details
Data Source	P2P Backend (`/api/customer/addresses`)
ViewModel	AddressViewModel
Validation	Google Places API
Default Address	Stored in @AppStorage

PaymentMethodsView.swift

Aspect	Details
Data Source	Stripe API via P2P Backend
ViewModel	PaymentService
Card Storage	Stripe (PCI compliant)
Local Storage	Last 4 digits only in @AppStorage

4.6 Additional Views

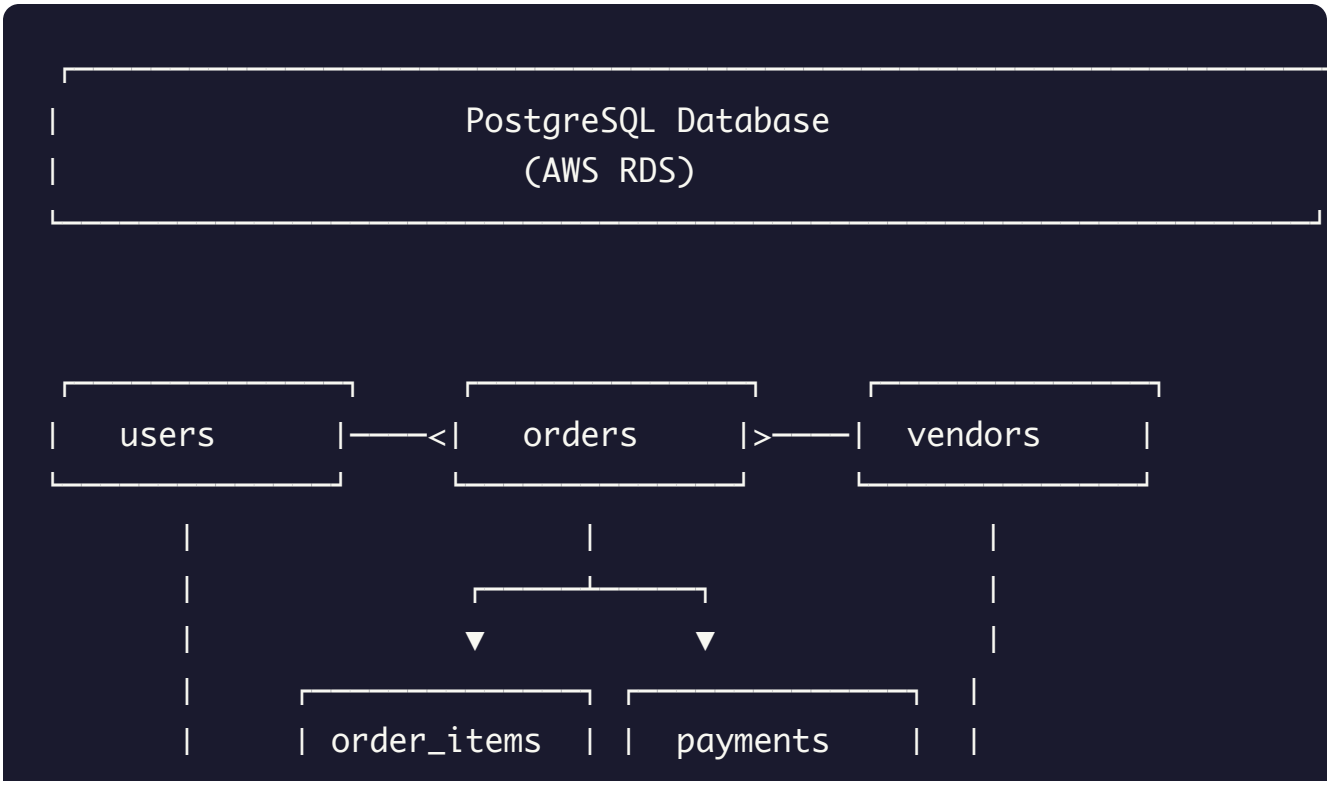
View	Primary Data Source	Firebase Usage	Local Storage

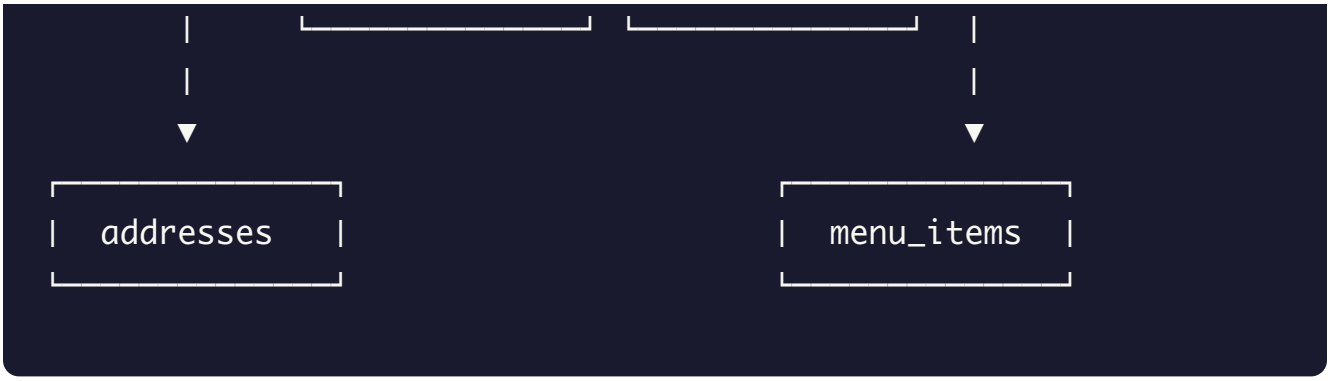
FavoritesView	P2P Backend	None	Favorites list
NotificationsView	P2P Backend + APNs	FCM tokens	Preferences
HelpCenterView	Static + P2P	None	None
RateDriverView	P2P Backend	None	None
TipDriverView	P2P + Stripe	None	None
DriverChatView	Firebase Firestore	Real-time messages	None
SearchView	P2P Backend	None	Recent searches
PromotionsView	P2P Backend	None	None

---

5. Database Schema

5.1 Complete Schema Overview





5.2 Table Definitions

users

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255),
    full_name VARCHAR(255) NOT NULL,
    phone VARCHAR(20),
    role VARCHAR(20) NOT NULL DEFAULT 'customer',
    google_id VARCHAR(255),
    apple_id VARCHAR(255),
    stripe_customer_id VARCHAR(255),
    is_active BOOLEAN DEFAULT TRUE,
    email_verified BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_login TIMESTAMP,

    CONSTRAINT valid_role CHECK (role IN ('customer', 'vendor', 'dr
);


 -- Indexes
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_google_id ON users(google_id);
```

```
CREATE INDEX idx_users_apple_id ON users(apple_id);
CREATE INDEX idx_users_role ON users(role);
```

vendors

```
CREATE TABLE vendors (
    id SERIAL PRIMARY KEY,
    vendor_id VARCHAR(50) UNIQUE NOT NULL,  -- Format: VEN-YYYYMM-X
    user_id INTEGER REFERENCES users(id),
    business_name VARCHAR(255) NOT NULL,
    description TEXT,
    cuisine_type VARCHAR(100),
    address TEXT NOT NULL,
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    phone VARCHAR(20),
    email VARCHAR(255),
    logo_url VARCHAR(500),
    banner_url VARCHAR(500),
    rating DECIMAL(2, 1) DEFAULT 0.0,
    total_ratings INTEGER DEFAULT 0,
    is_open BOOLEAN DEFAULT TRUE,
    is_active BOOLEAN DEFAULT TRUE,
    is_featured BOOLEAN DEFAULT FALSE,
    commission_rate DECIMAL(4, 2) DEFAULT 15.00,
    minimum_order DECIMAL(10, 2) DEFAULT 0.00,
    delivery_fee DECIMAL(10, 2) DEFAULT 1.00,
    estimated_prep_time INTEGER DEFAULT 30,  -- minutes
    operating_hours JSONB,
    stripe_account_id VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
 -- Indexes
CREATE INDEX idx_vendors_vendor_id ON vendors(vendor_id);
CREATE INDEX idx_vendors_location ON vendors(latitude, longitude);
CREATE INDEX idx_vendors_cuisine ON vendors(cuisine_type);
CREATE INDEX idx_vendors_active ON vendors(is_active, is_open);
```

menu_items

```
 CREATE TABLE menu_items (
    id SERIAL PRIMARY KEY,
    vendor_id INTEGER REFERENCES vendors(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2) NOT NULL,
    category VARCHAR(100),
    image_url VARCHAR(500),
    is_available BOOLEAN DEFAULT TRUE,
    is_featured BOOLEAN DEFAULT FALSE,
    prep_time INTEGER DEFAULT 15,  -- minutes
    calories INTEGER,
    is_vegetarian BOOLEAN DEFAULT FALSE,
    is_vegan BOOLEAN DEFAULT FALSE,
    is_gluten_free BOOLEAN DEFAULT FALSE,
    is_spicy BOOLEAN DEFAULT FALSE,
    spice_level INTEGER DEFAULT 0,
    allergens JSONB,
    customizations JSONB,
    sort_order INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


 -- Indexes
CREATE INDEX idx_menu_items_vendor ON menu_items(vendor_id);
```

```
CREATE INDEX idx_menu_items_category ON menu_items(vendor_id, categ
CREATE INDEX idx_menu_items_available ON menu_items(vendor_id, is_a
```

orders

```
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    order_id VARCHAR(50) UNIQUE NOT NULL,  -- Format: ORD-YYYYMMDD-
    customer_id INTEGER REFERENCES users(id),
    vendor_id INTEGER REFERENCES vendors(id),
    driver_id INTEGER REFERENCES users(id),

    -- Status tracking
    status VARCHAR(30) NOT NULL DEFAULT 'pending',
    status_history JSONB DEFAULT '[]',

    -- Pricing
    subtotal DECIMAL(10, 2) NOT NULL,
    tax_amount DECIMAL(10, 2) NOT NULL,
    delivery_fee DECIMAL(10, 2) NOT NULL DEFAULT 1.00,
    tip_amount DECIMAL(10, 2) DEFAULT 0.00,
    discount_amount DECIMAL(10, 2) DEFAULT 0.00,
    total_amount DECIMAL(10, 2) NOT NULL,

    -- Promo code
    promo_code_id INTEGER REFERENCES promo_codes(id),
    promo_code VARCHAR(50),

    -- Delivery info
    delivery_address_id INTEGER REFERENCES addresses(id),
    delivery_address TEXT NOT NULL,
    delivery_latitude DECIMAL(10, 8),
    delivery_longitude DECIMAL(11, 8),
    delivery_instructions TEXT,

    -- Timing
    estimated_prep_time INTEGER,  -- minutes
```

```
    estimated_delivery_time TIMESTAMP,
    actual_delivery_time TIMESTAMP,

    -- Metadata
    special_instructions TEXT,
    is_scheduled BOOLEAN DEFAULT FALSE,
    scheduled_time TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT valid_status CHECK (status IN (
        'pending', 'confirmed', 'preparing', 'ready_for_pickup',
        'picked_up', 'out_for_delivery', 'delivered', 'cancelled',
    ))
);


 -- Indexes
CREATE INDEX idx_orders_order_id ON orders(order_id);
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_vendor ON orders(vendor_id);
CREATE INDEX idx_orders_driver ON orders(driver_id);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created ON orders(created_at DESC);
```

order_items

```
 CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INTEGER REFERENCES orders(id) ON DELETE CASCADE,
    menu_item_id INTEGER REFERENCES menu_items(id),
    name VARCHAR(255) NOT NULL,
    quantity INTEGER NOT NULL DEFAULT 1,
    unit_price DECIMAL(10, 2) NOT NULL,
    total_price DECIMAL(10, 2) NOT NULL,
    customizations JSONB,
    special_instructions TEXT,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


 -- Indexes
CREATE INDEX idx_order_items_order ON order_items(order_id);
```

addresses

```
 CREATE TABLE addresses (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    label VARCHAR(50),  -- 'Home', 'Work', 'Other'
    street_address VARCHAR(255) NOT NULL,
    apt_suite VARCHAR(50),
    city VARCHAR(100) NOT NULL,
    state VARCHAR(50) NOT NULL,
    zip_code VARCHAR(20) NOT NULL,
    country VARCHAR(50) DEFAULT 'USA',
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    is_default BOOLEAN DEFAULT FALSE,
    delivery_instructions TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


 -- Indexes
CREATE INDEX idx_addresses_user ON addresses(user_id);
CREATE INDEX idx_addresses_default ON addresses(user_id, is_default
```

payments

```
CREATE TABLE payments (
    id SERIAL PRIMARY KEY,
    payment_id VARCHAR(50) UNIQUE NOT NULL,
    order_id INTEGER REFERENCES orders(id),
    user_id INTEGER REFERENCES users(id),

    -- Stripe data
    stripe_payment_intent_id VARCHAR(255),
    stripe_charge_id VARCHAR(255),
    stripe_refund_id VARCHAR(255),

    -- Payment details
    amount DECIMAL(10, 2) NOT NULL,
    currency VARCHAR(3) DEFAULT 'usd',
    status VARCHAR(30) NOT NULL DEFAULT 'pending',
    payment_method VARCHAR(30),  -- 'card', 'ach', 'apple_pay', 'go
    card_last_four VARCHAR(4),
    card_brand VARCHAR(20),

    -- Metadata
    failure_reason TEXT,
    refund_reason TEXT,
    metadata JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT valid_payment_status CHECK (status IN (
        'pending', 'processing', 'succeeded', 'failed', 'refunded',
    ))
);


 -- Indexes
CREATE INDEX idx_payments_order ON payments(order_id);
CREATE INDEX idx_payments_user ON payments(user_id);
CREATE INDEX idx_payments_stripe ON payments(stripe_payment_intent_
CREATE INDEX idx_payments_status ON payments(status);
```

drivers

```
CREATE TABLE drivers (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) UNIQUE,
    driver_id VARCHAR(50) UNIQUE NOT NULL,  -- Format: DRV-YYYYMM-X
    vehicle_type VARCHAR(50),
    vehicle_make VARCHAR(50),
    vehicle_model VARCHAR(50),
    vehicle_color VARCHAR(30),
    license_plate VARCHAR(20),
    license_number VARCHAR(50),
    insurance_info JSONB,
    is_available BOOLEAN DEFAULT FALSE,
    is_active BOOLEAN DEFAULT TRUE,
    is_verified BOOLEAN DEFAULT FALSE,
    current_latitude DECIMAL(10, 8),
    current_longitude DECIMAL(11, 8),
    last_location_update TIMESTAMP,
    rating DECIMAL(2, 1) DEFAULT 5.0,
    total_ratings INTEGER DEFAULT 0,
    total_deliveries INTEGER DEFAULT 0,
    stripe_account_id VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


 -- Indexes
CREATE INDEX idx_drivers_user ON drivers(user_id);
CREATE INDEX idx_drivers_driver_id ON drivers(driver_id);
CREATE INDEX idx_drivers_available ON drivers(is_available, is_acti
CREATE INDEX idx_drivers_location ON drivers(current_latitude, curr
```

promo_codes

```
CREATE TABLE promo_codes (
    id SERIAL PRIMARY KEY,
    code VARCHAR(50) UNIQUE NOT NULL,
    description TEXT,
    discount_type VARCHAR(20) NOT NULL,  -- 'percentage', 'fixed'
    discount_value DECIMAL(10, 2) NOT NULL,
    minimum_order DECIMAL(10, 2) DEFAULT 0.00,
    maximum_discount DECIMAL(10, 2),
    usage_limit INTEGER,
    used_count INTEGER DEFAULT 0,
    per_user_limit INTEGER DEFAULT 1,
    valid_from TIMESTAMP NOT NULL,
    valid_until TIMESTAMP NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    applies_to VARCHAR(20) DEFAULT 'all',  -- 'all', 'first_order',
    vendor_ids JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT valid_discount_type CHECK (discount_type IN ('percen
);


 -- Indexes
CREATE INDEX idx_promo_codes_code ON promo_codes(code);
CREATE INDEX idx_promo_codes_active ON promo_codes(is_active, valid
```

promo_code_usage

```
CREATE TABLE promo_code_usage (
    id SERIAL PRIMARY KEY,
    promo_code_id INTEGER REFERENCES promo_codes(id),
    user_id INTEGER REFERENCES users(id),
    order_id INTEGER REFERENCES orders(id),
    discount_applied DECIMAL(10, 2) NOT NULL,
    used_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    UNIQUE(promo_code_id, user_id, order_id)
);
```

reviews

```
CREATE TABLE reviews (
    id SERIAL PRIMARY KEY,
    order_id INTEGER REFERENCES orders(id),
    customer_id INTEGER REFERENCES users(id),
    vendor_id INTEGER REFERENCES vendors(id),
    driver_id INTEGER REFERENCES users(id),

    -- Ratings
    food_rating INTEGER CHECK (food_rating BETWEEN 1 AND 5),
    delivery_rating INTEGER CHECK (delivery_rating BETWEEN 1 AND 5)
    overall_rating INTEGER CHECK (overall_rating BETWEEN 1 AND 5),

    -- Review content
    food_review TEXT,
    delivery_review TEXT,

    -- Metadata
    is_public BOOLEAN DEFAULT TRUE,
    vendor_response TEXT,
    vendor_responded_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


-- Indexes
CREATE INDEX idx_reviews_order ON reviews(order_id);
CREATE INDEX idx_reviews_vendor ON reviews(vendor_id);
CREATE INDEX idx_reviews_driver ON reviews(driver_id);
```

notifications

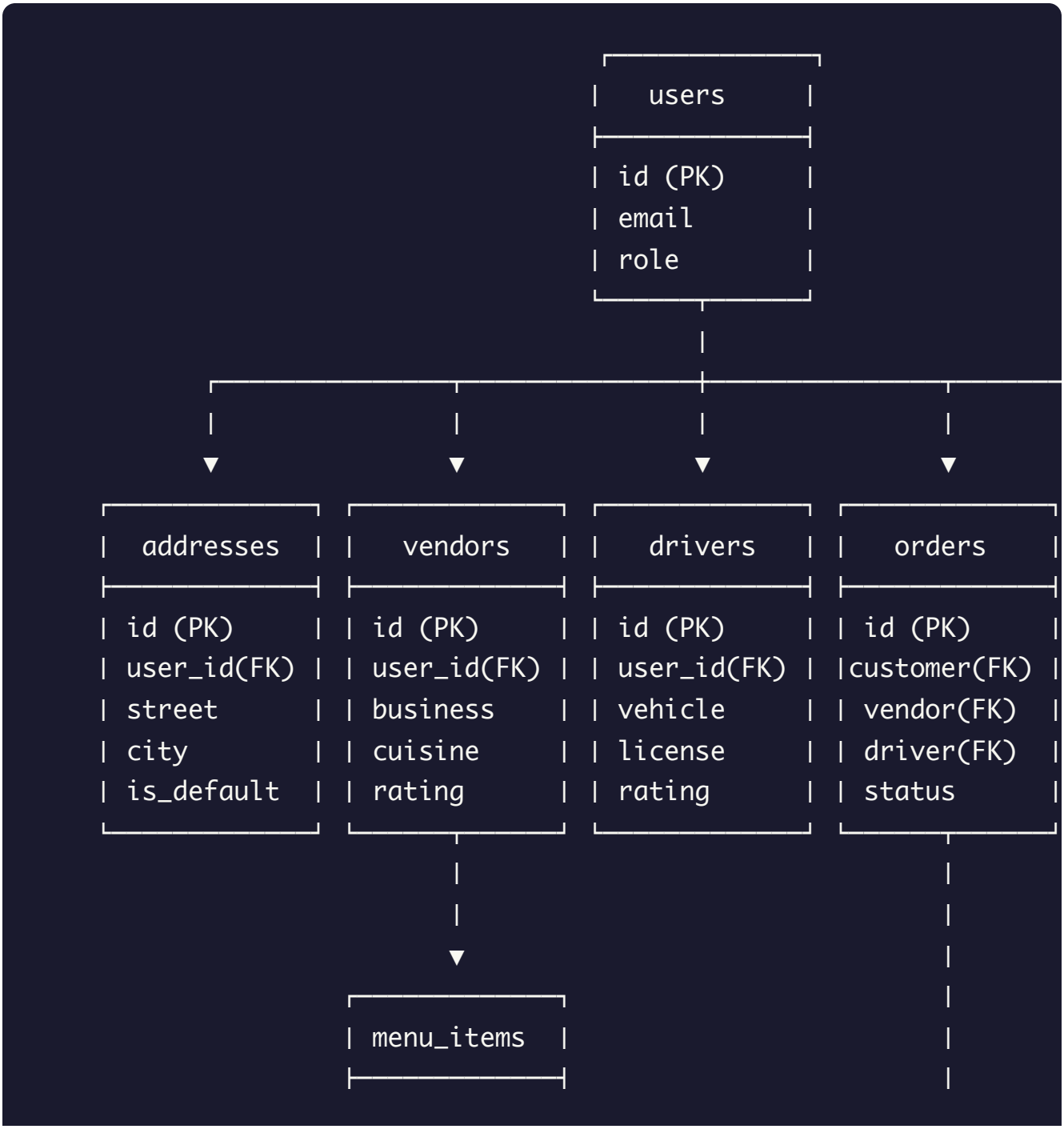
```
CREATE TABLE notifications (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    type VARCHAR(50) NOT NULL,
    title VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    data JSONB,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


-- Indexes
CREATE INDEX idx_notifications_user ON notifications(user_id, is_re
CREATE INDEX idx_notifications_created ON notifications(created_at
```

journal_entries (Double-Entry Accounting)

```
CREATE TABLE journal_entries (
    id SERIAL PRIMARY KEY,
    entry_id VARCHAR(50) UNIQUE NOT NULL,
    order_id INTEGER REFERENCES orders(id),
    payment_id INTEGER REFERENCES payments(id),
    entry_type VARCHAR(50) NOT NULL,
    description TEXT,
    entries JSONB NOT NULL,  -- Array of {account, debit, credit}
    total_debit DECIMAL(12, 2) NOT NULL,
    total_credit DECIMAL(12, 2) NOT NULL,
    is_balanced BOOLEAN GENERATED ALWAYS AS (total_debit = total_cr
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_by INTEGER REFERENCES users(id)
);
```

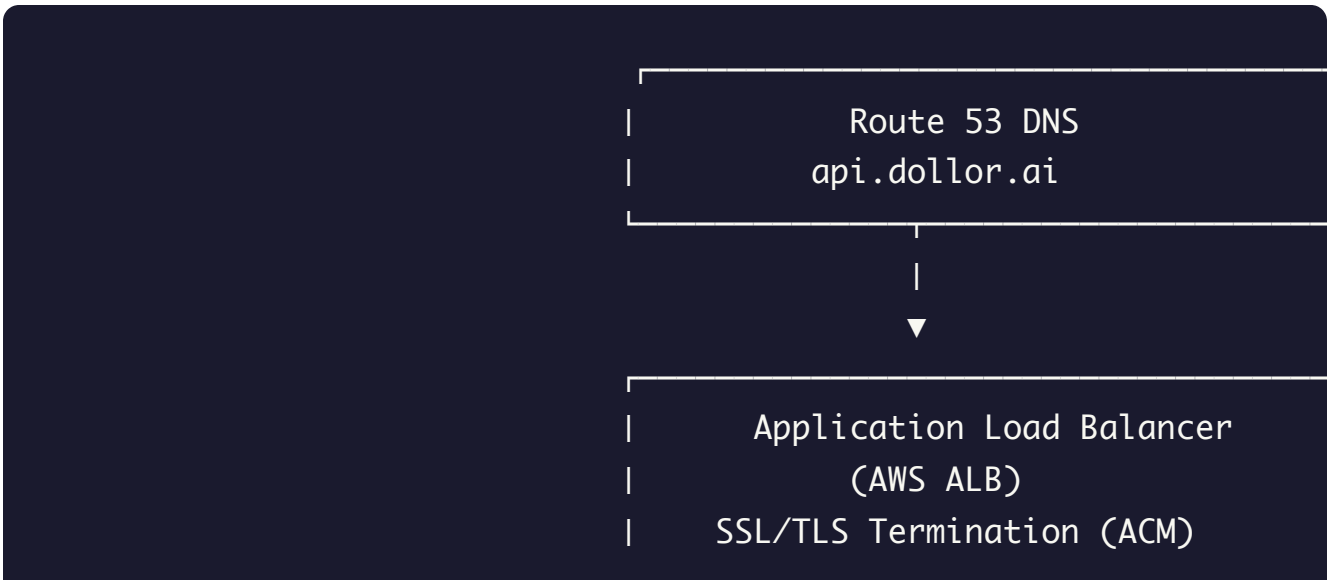
```
 -- Example entries JSONB structure:
-- [
--   {"account": "cash", "debit": 25.00, "credit": 0},
--   {"account": "revenue", "debit": 0, "credit": 22.50},
--   {"account": "tax_payable", "debit": 0, "credit": 1.50},
--   {"account": "delivery_fee", "debit": 0, "credit": 1.00}
-- ]
```

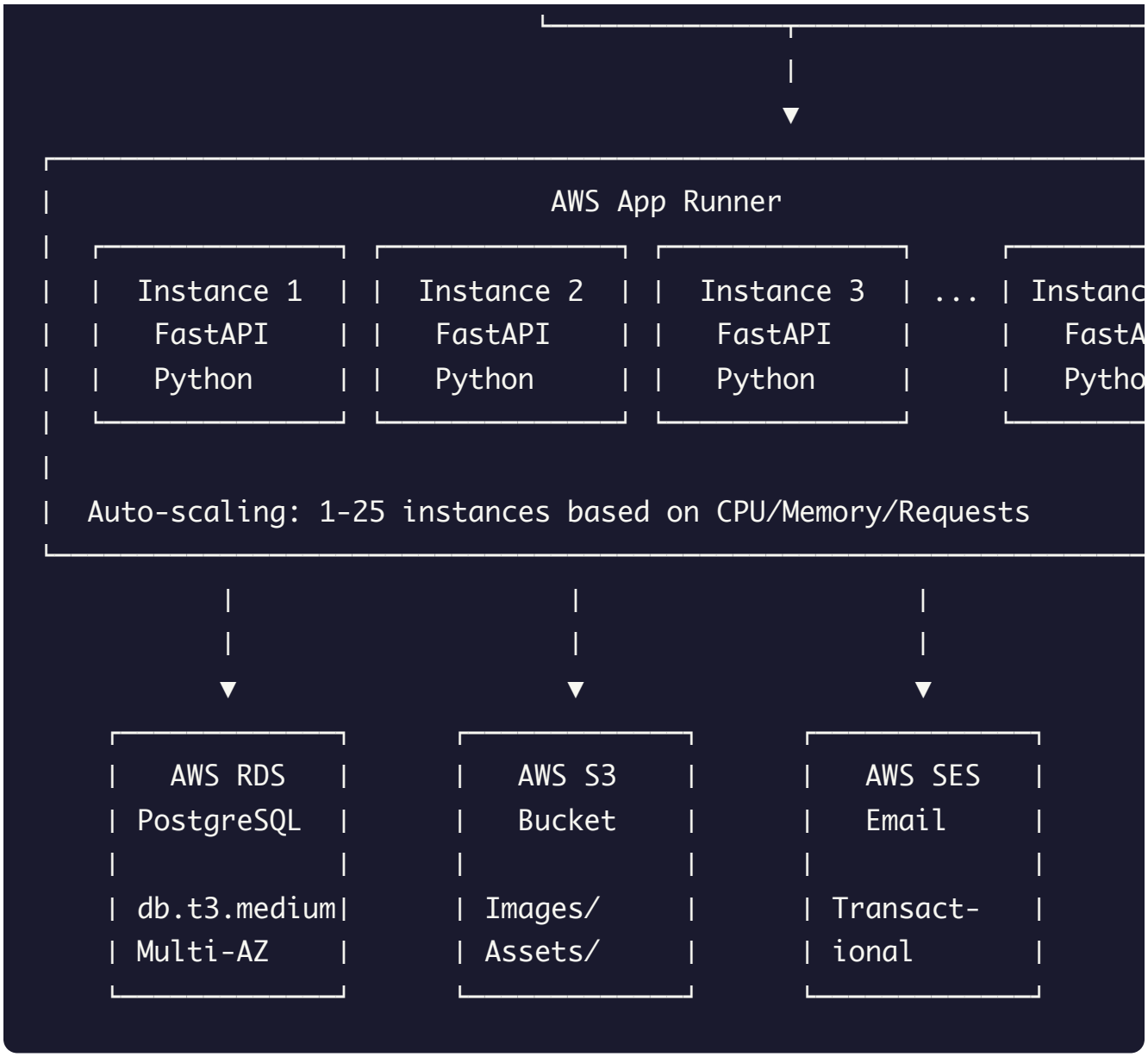
5.3 Entity Relationship Diagram



6. AWS Infrastructure

6.1 Architecture Overview





6.2 AWS Services Used

Service	Purpose	Configuration
App Runner	Application hosting	Auto-scale 1-25 instances, 1 vCPU, 2GB RAM per instance
RDS	PostgreSQL database	db.t3.medium, Multi-AZ, automated backups
S3	File storage	Restaurant images, user uploads, assets

SES	Transactional email	Password reset, order confirmations, receipts
CloudWatch	Monitoring & logs	Metrics, alarms, log retention 30 days
Secrets Manager	Credentials	Database passwords, API keys, Stripe keys
ACM	SSL certificates	Auto-renewed certificates for api.dollor.ai
Route 53	DNS management	Domain routing, health checks
IAM	Access control	Role-based permissions, service accounts

6.3 App Runner Configuration

```
version: 1.0
runtime: python3
build:
  commands:
    build:
      - pip install -r requirements.txt
run:
  command: uvicorn main_new:app --host 0.0.0.0 --port 8080
  network:
    port: 8080
  env:
    - name: DATABASE_URL
      value-from: "arn:aws:secretsmanager:us-east-1:xxx:secret:doll
    - name: STRIPE_SECRET_KEY
```

```
      value-from: "arn:aws:secretsmanager:us-east-1:xxx:secret:doll
    - name: JWT_SECRET
      value-from: "arn:aws:secretsmanager:us-east-1:xxx:secret:doll
```

```
auto_scaling:
  min_size: 1
  max_size: 25
  max_concurrency: 100  # requests per instance before scaling


 health_check:
  protocol: HTTP
  path: /health
  interval: 10
  timeout: 5
  healthy_threshold: 1
  unhealthy_threshold: 5
```

6.4 RDS Configuration

```
 Instance Class: db.t3.medium
Engine: PostgreSQL 15
Storage: 100 GB gp3 (auto-scaling to 500 GB)
Multi-AZ: Enabled
Backup Retention: 7 days
Encryption: AES-256 at rest
SSL: Required for connections
```

```
 Connection Pooling: PgBouncer (built into app)
Max Connections: 100 per instance
```

6.5 S3 Bucket Structure

```
 s3://dollor-assets/
├── restaurants/
|   ├── logos/
|   |   └── {vendor_id}/logo.jpg
|   ├── banners/
|   |   └── {vendor_id}/banner.jpg
|   └── menu/
|       └── {vendor_id}/{item_id}.jpg
├── users/
|   └── profiles/
|       └── {user_id}/avatar.jpg
├── drivers/
|   └── documents/
|       └── {driver_id}/
|           ├── license.jpg
|           └── insurance.jpg
└── receipts/
    └── {order_id}/receipt.pdf
```

6.6 Environment Variables

Variable	Source	Description
`DATABASE_URL`	Secrets Manager	PostgreSQL connection string
`STRIPE_SECRET_KEY`	Secrets Manager	Stripe API secret key
`STRIPE_WEBHOOK_SECRET`	Secrets Manager	Stripe webhook signing secret
`JWT_SECRET`	Secrets Manager	JWT token signing key

`AWS_S3_BUCKET`	Environment	S3 bucket name
`AWS_REGION`	Environment	AWS region (us-east-1)
`APNS_KEY_ID`	Secrets Manager	Apple Push Notification key
`APNS_TEAM_ID`	Secrets Manager	Apple Developer Team ID
`GOOGLE_MAPS_API_KEY`	Secrets Manager	Google Maps server key
`FIREBASE_PROJECT_ID`	Environment	Firebase project for fallback

6.7 Monitoring & Alerts

CloudWatch Alarms: - CPU utilization > 80% for 5 minutes - Memory utilization > 85% for 5 minutes - Error rate > 5% for 2 minutes - Response time p99 > 3 seconds - Database connections > 80 for 5 minutes

Log Groups: - `/aws/apprunner/dollor-api` - Application logs - `/aws/rds/dollor-db` - Database logs - `/dollor/stripe-webhooks` - Payment webhooks

---

7. API Endpoints

7.1 Authentication Endpoints

Method	Endpoint	Description	Auth
POST	`/api/customer/register`	Register new customer	No
POST	`/api/customer/login`	Email/password login	No
POST	`/api/customer/google-auth`	Google OAuth login	No

POST	`/api/customer/apple-auth`	Apple Sign-In	No
POST	`/api/customer/logout`	Logout (invalidate token)	Yes
POST	`/api/customer/request-password-reset`	Request reset code	No
POST	`/api/customer/confirm-password-reset`	Reset with code	No
GET	`/api/customer/profile`	Get customer profile	Yes
PUT	`/api/customer/profile`	Update profile	Yes

7.2 Restaurant Endpoints

Method	Endpoint	Description	Auth
GET	`/api/restaurants`	List all restaurants	No
GET	`/api/restaurants/{id}`	Get restaurant detail + menu	No
GET	`/api/restaurants/nearby`	Get nearby restaurants	No
GET	`/api/restaurants/search`	Search restaurants	No
GET	`/api/restaurants/{id}/menu`	Get menu items	No
GET	`/api/restaurants/featured`	Get featured restaurants	No
GET	`/api/restaurants/cuisines`	Get cuisine categories	No

7.3 Order Endpoints

Method	Endpoint	Description	Auth

POST	`/api/orders`	Create new order	Yes
GET	`/api/customer/orders`	Get customer's orders	Yes
GET	`/api/orders/{id}`	Get order details	Yes
GET	`/api/orders/{id}/status`	Get order status	Yes
PUT	`/api/orders/{id}/cancel`	Cancel order	Yes
POST	`/api/orders/{id}/reorder`	Reorder previous order	Yes
GET	`/api/orders/{id}/tracking`	Get tracking info	Yes

7.4 Payment Endpoints

Method	Endpoint	Description	Auth
POST	`/api/payments/create-intent`	Create Stripe payment intent	Yes
POST	`/api/payments/confirm`	Confirm payment	Yes
GET	`/api/payments/methods`	List saved payment methods	Yes
POST	`/api/payments/methods`	Add payment method	Yes
DELETE	`/api/payments/methods/{id}`	Delete payment method	Yes
POST	`/api/payments/tip`	Add tip to order	Yes
POST	`/api/webhooks/stripe`	Stripe webhook handler	No*

7.5 Address Endpoints

Method	Endpoint	Description	Auth
GET	`/api/customer/addresses`	List saved addresses	Yes
POST	`/api/customer/addresses`	Add new address	Yes
PUT	`/api/customer/addresses/{id}`	Update address	Yes
DELETE	`/api/customer/addresses/{id}`	Delete address	Yes
PUT	`/api/customer/addresses/{id}/default`	Set as default	Yes
POST	`/api/addresses/validate`	Validate address	Yes

7.6 Promo Code Endpoints

Method	Endpoint	Description	Auth
POST	`/api/promo/validate`	Validate promo code	Yes
POST	`/api/promo/apply`	Apply to order	Yes
GET	`/api/promo/available`	Get available promos	Yes

7.7 Review & Rating Endpoints

Method	Endpoint	Description	Auth
POST	`/api/orders/{id}/review`	Submit order review	Yes
GET	`/api/restaurants/{id}/reviews`	Get restaurant reviews	No

POST	`/api/orders/{id}/rate-driver`	Rate delivery driver	Yes

7.8 Driver Endpoints (Delivery App)

Method	Endpoint	Description	Auth
POST	`/api/driver/register`	Register as driver	No
POST	`/api/driver/login`	Driver login	No
GET	`/api/driver/profile`	Get driver profile	Yes
PUT	`/api/driver/location`	Update location	Yes
PUT	`/api/driver/availability`	Toggle availability	Yes
GET	`/api/driver/orders`	Get assigned orders	Yes
PUT	`/api/driver/orders/{id}/accept`	Accept order	Yes
PUT	`/api/driver/orders/{id}/pickup`	Mark picked up	Yes
PUT	`/api/driver/orders/{id}/deliver`	Mark delivered	Yes

7.9 Vendor Endpoints (Restaurant App)

Method	Endpoint	Description	Auth
POST	`/api/vendor/login`	Vendor login	No
GET	`/api/vendor/orders`	Get incoming orders	Yes
PUT	`/api/vendor/orders/{id}/accept`	Accept order	Yes

PUT	`/api/vendor/orders/{id}/ready`	Mark ready for pickup	Yes
PUT	`/api/vendor/orders/{id}/cancel`	Cancel order	Yes
GET	`/api/vendor/menu`	Get menu items	Yes
POST	`/api/vendor/menu`	Add menu item	Yes
PUT	`/api/vendor/menu/{id}`	Update menu item	Yes
DELETE	`/api/vendor/menu/{id}`	Delete menu item	Yes
PUT	`/api/vendor/menu/{id}/availability`	Toggle availability	Yes
GET	`/api/vendor/analytics`	Get sales analytics	Yes

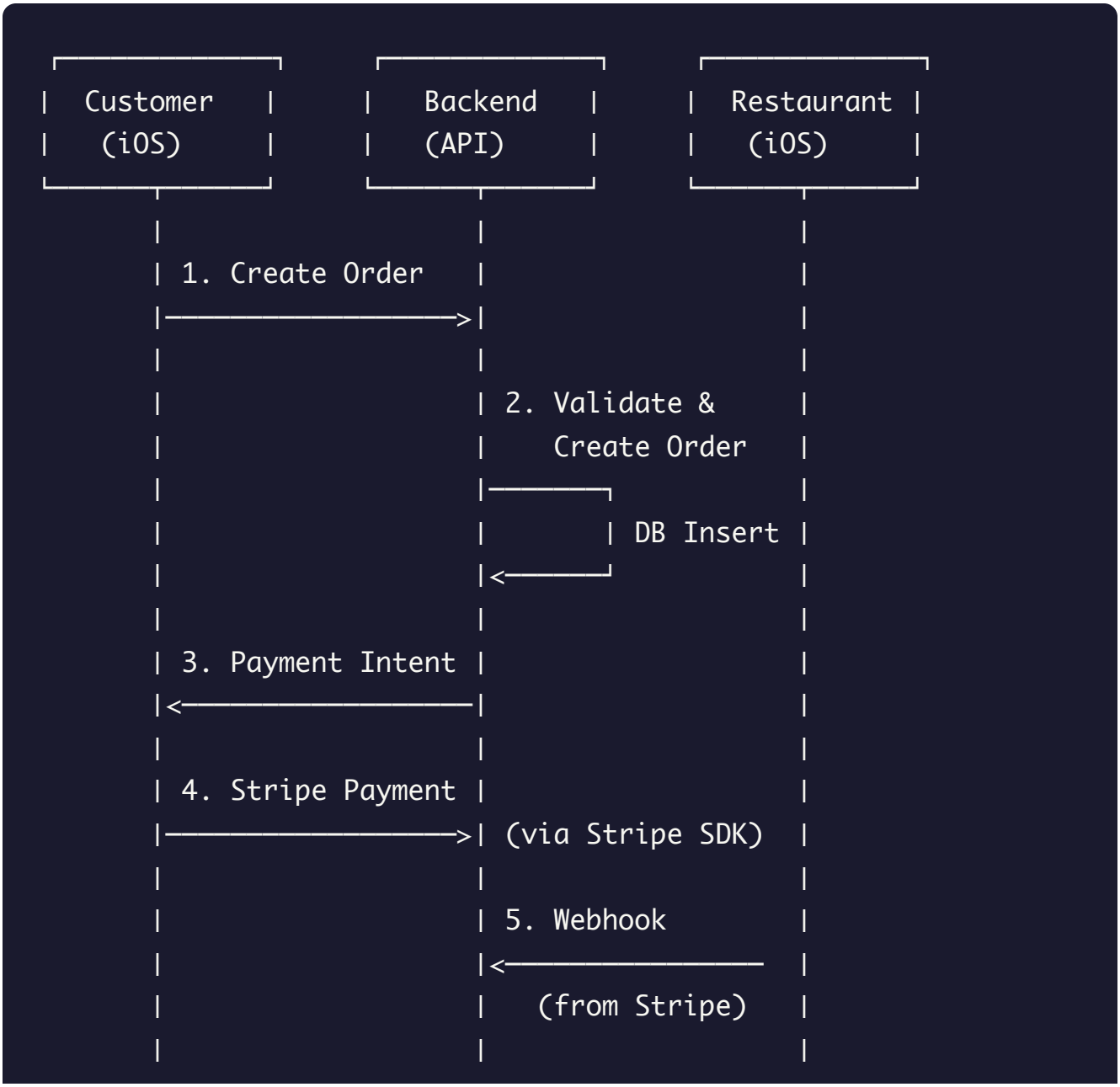
7.10 Admin Endpoints

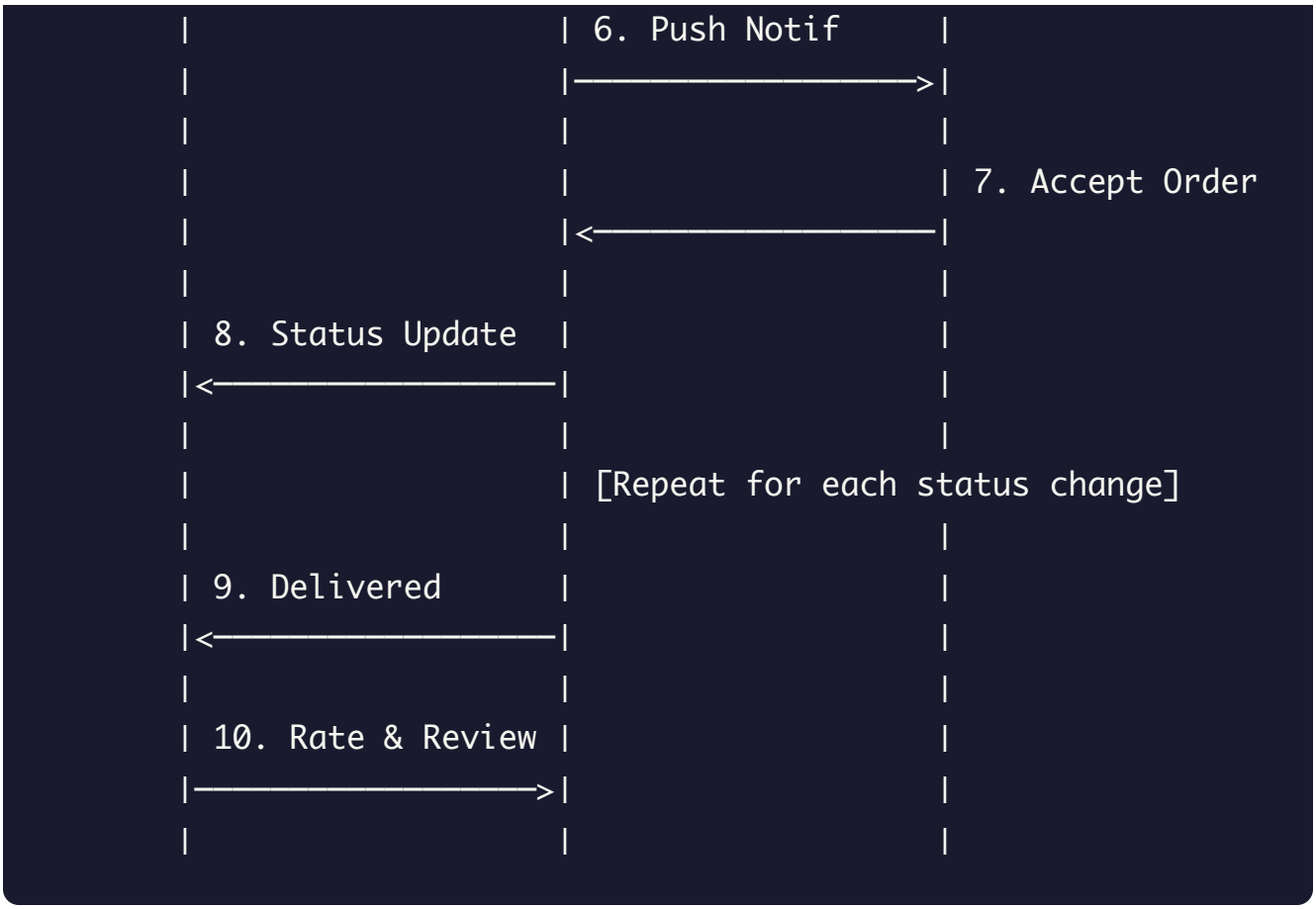
Method	Endpoint	Description	Auth
GET	`/api/admin/dashboard`	Dashboard stats	Admin
GET	`/api/admin/users`	List all users	Admin
GET	`/api/admin/orders`	List all orders	Admin
GET	`/api/admin/vendors`	List all vendors	Admin
GET	`/api/admin/drivers`	List all drivers	Admin
POST	`/api/admin/promo-codes`	Create promo code	Admin
GET	`/api/admin/analytics`	Platform analytics	Admin

GET	`/api/admin/reports/revenue`	Revenue reports	Admin
POST	`/api/admin/vendors/{id}/approve`	Approve vendor	Admin
POST	`/api/admin/drivers/{id}/verify`	Verify driver	Admin

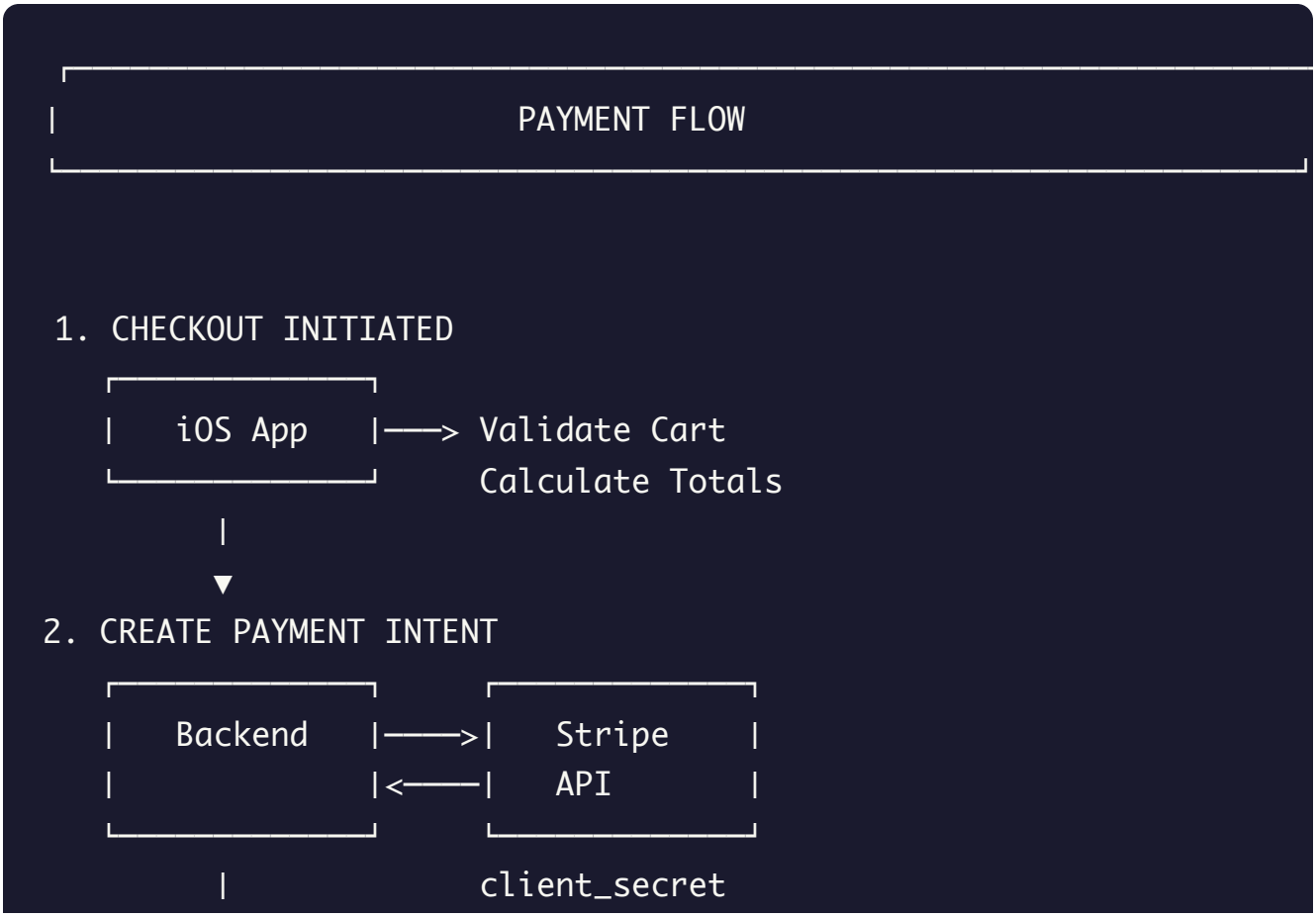
8. Data Flow Diagrams

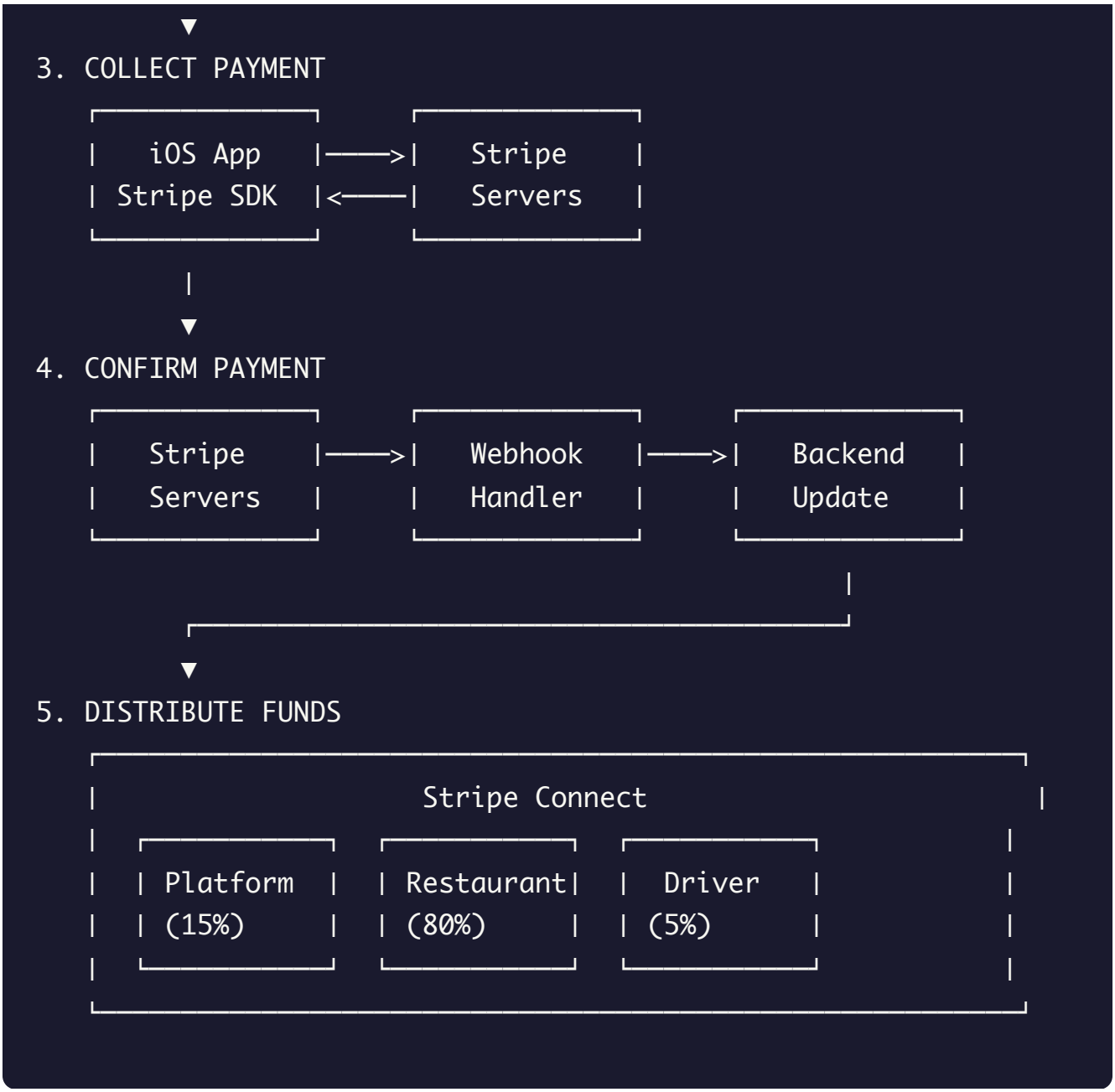
8.1 Order Lifecycle



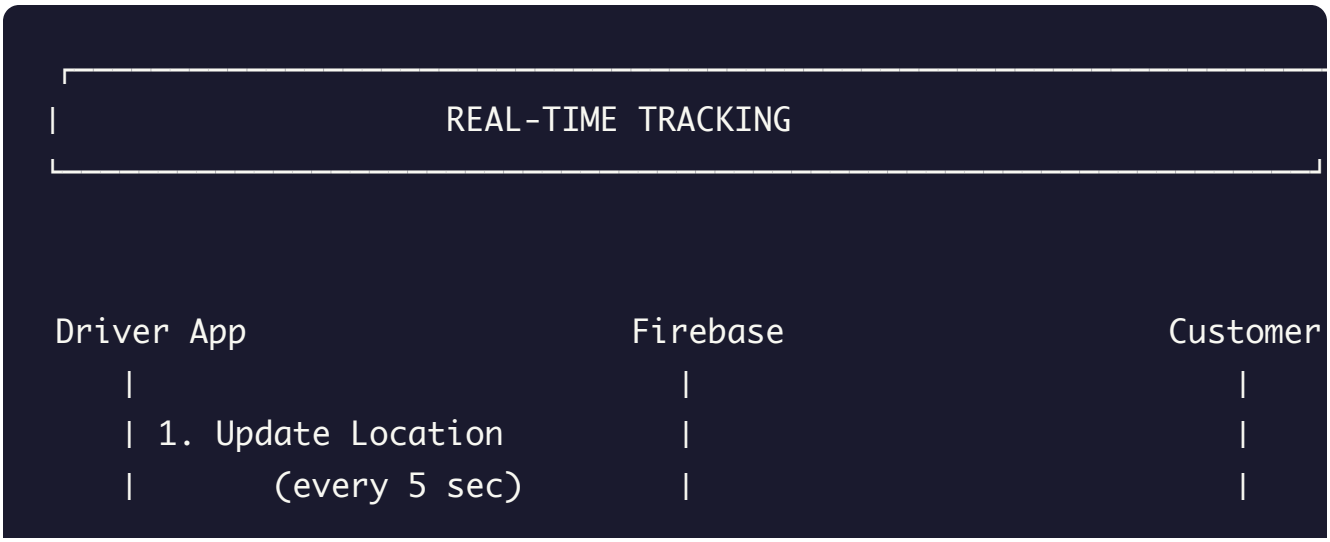


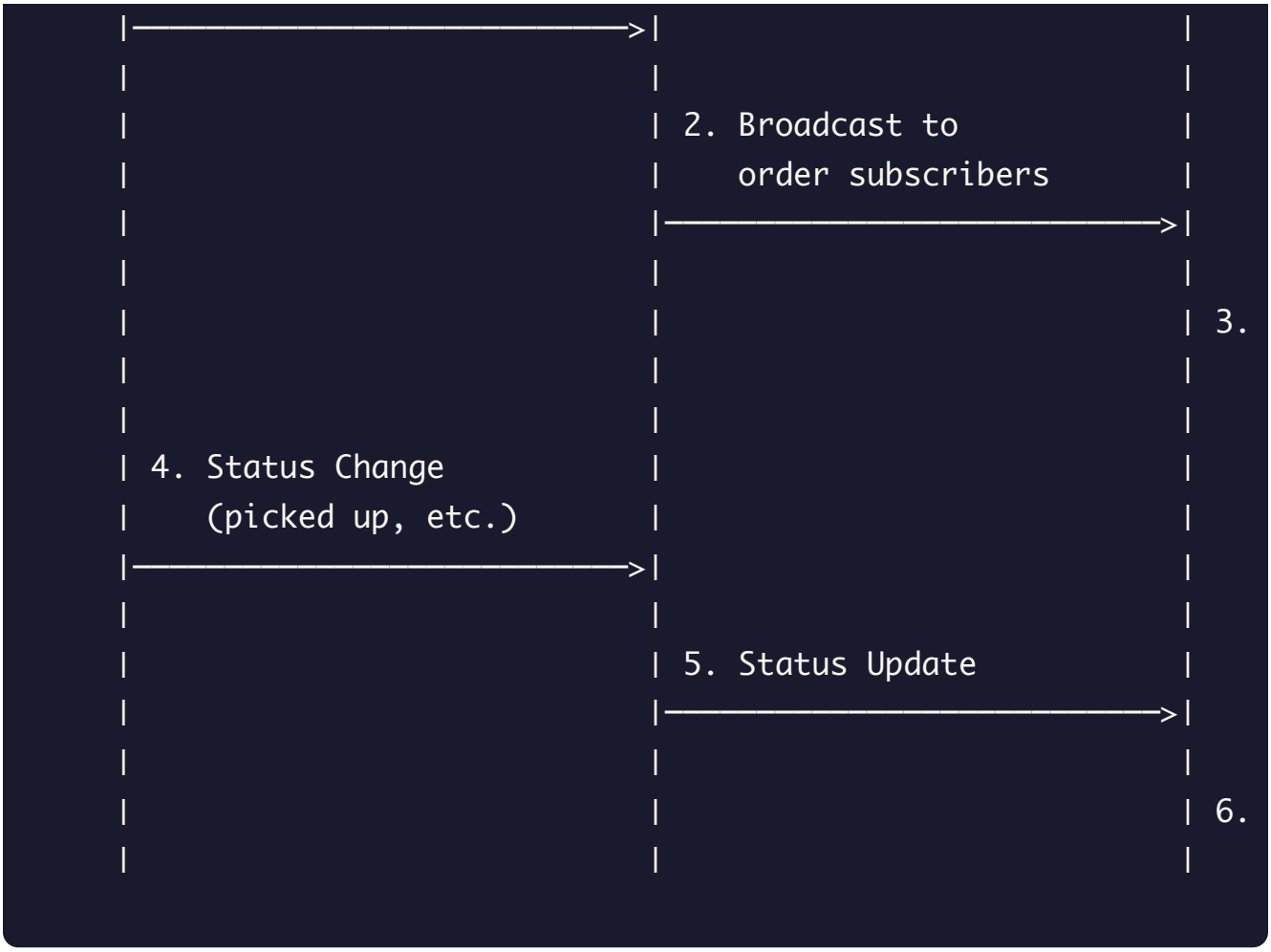
8.2 Payment Flow





8.3 Real-time Tracking

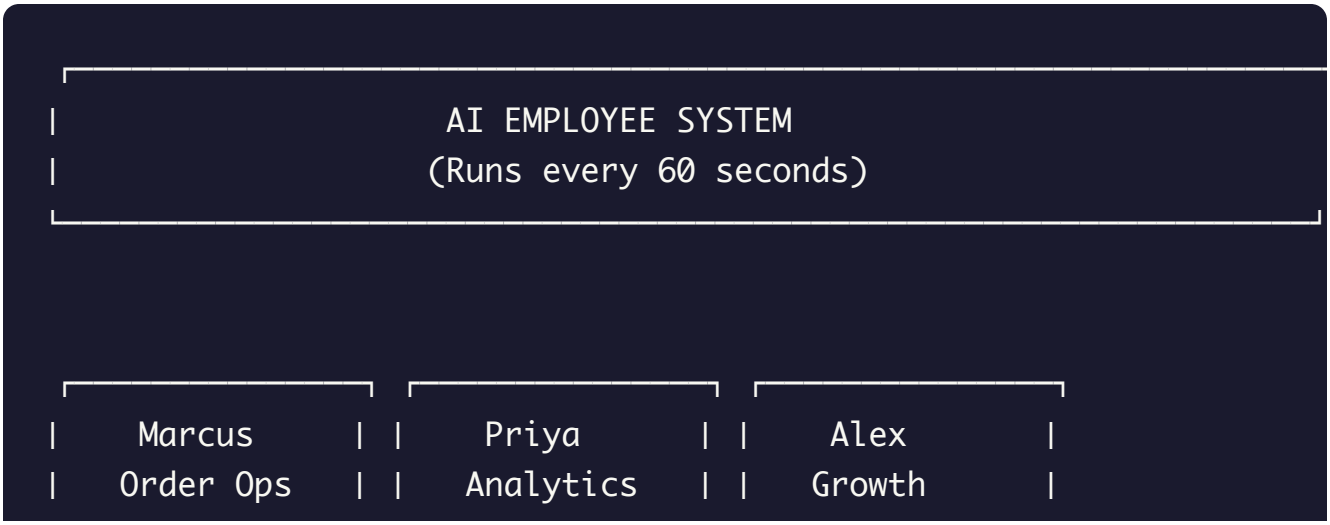


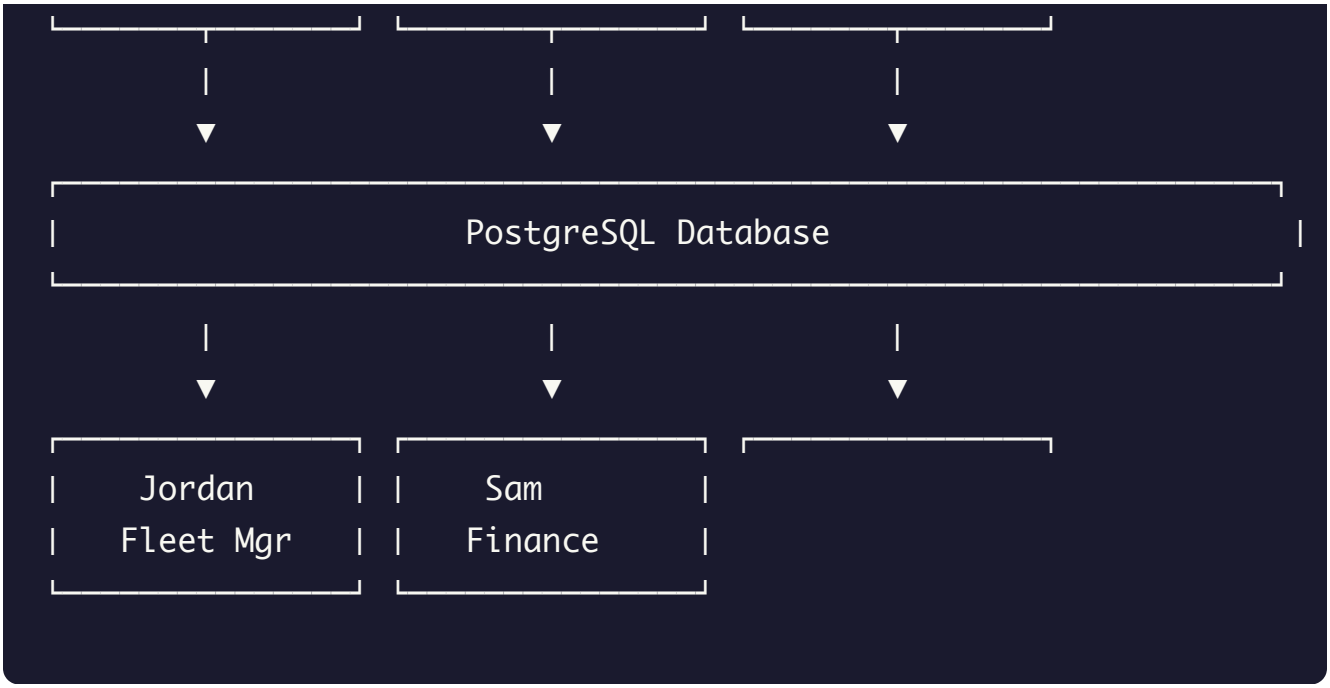


9. AI Employee Automation

9.1 Overview

Dollor.ai uses 5 AI Employees that run continuously to automate platform operations:





9.2 AI Employee Roles

Marcus - Order Operations Manager

Responsibilities: - Monitor order flow and identify bottlenecks - Auto-assign drivers to orders - Handle delayed order escalations - Coordinate between restaurants and drivers

Automated Actions:

```
async def marcus_duties():
    # Check for orders without drivers (>10 min)
    unassigned = await get_unassigned_orders(threshold_minutes=10)
    for order in unassigned:
        await auto_assign_nearest_driver(order)
        await notify_customer_delay(order)


     # Check for preparing orders (>30 min)
    slow_orders = await get_slow_preparing_orders(threshold_minutes
    for order in slow_orders:
        await ping_restaurant(order.vendor_id)
        await update_customer_eta(order)
```

Priya - Analytics & Insights

Responsibilities: - Generate real-time business metrics - Identify trends and anomalies - Create daily/weekly reports - Monitor KPIs

Automated Reports: - Hourly order volume - Restaurant performance scores - Driver efficiency ratings - Customer retention metrics

Alex - Growth Marketing

Responsibilities: - Identify inactive customers - Trigger re-engagement campaigns - A/B test promotions - Optimize promo code distribution

Automated Campaigns:

```
async def alex_campaigns():
    # Customers inactive for 7+ days
    inactive = await get_inactive_customers(days=7)
    for customer in inactive:
        await send_winback_email(customer, promo="COMEBACK10")


     # First-time order follow-up
    new_customers = await get_first_orders_today()
    for customer in new_customers:
        await schedule_followup_email(customer, delay_hours=24)
```

Jordan - Fleet Manager

Responsibilities: - Monitor driver availability - Optimize driver distribution - Handle driver issues - Manage peak hour allocation

Real-time Actions:

```
async def jordan_fleet_management():
    # Check driver distribution across zones
    zones = await get_zone_driver_counts()
    for zone in zones:
```

```
        if zone.drivers < zone.min_required:
            await alert_operations_team(zone)
            await incentivize_nearby_drivers(zone)


    # Monitor driver response times
    slow_drivers = await get_slow_response_drivers()
    for driver in slow_drivers:
        await send_performance_alert(driver)
```

Sam - Finance Controller

Responsibilities: - Process daily settlements - Generate accounting entries - Reconcile payments - Handle refunds

Daily Tasks:

```
async def sam_daily_settlement():
    # Calculate restaurant payouts
    for vendor in await get_active_vendors():
        orders = await get_vendor_orders_yesterday(vendor.id)
        payout = sum(o.vendor_amount for o in orders)
        await create_stripe_transfer(vendor.stripe_account_id, payo
        await create_journal_entry(
            type="vendor_payout",
            entries=[
                {"account": "vendor_payable", "debit": payout},
                {"account": "cash", "credit": payout}
            ]
        )
```

9.3 AI Employee Schedule

Employee	Frequency	Time	Duration

Marcus	Every 60s	24/7	~5s per run
Priya	Every 5min	24/7	~30s per run
Alex	Every 1hr	24/7	~2min per run
Jordan	Every 60s	Peak hours	~5s per run
Sam	Daily	3:00 AM	~10min

---

10. App Store Readiness

10.1 Completed Fixes

Category	Issue	Status
Critical	Force unwrap crashes	Fixed
Critical	Hardcoded test promo codes	Removed
High	Unguarded debug prints	Wrapped in #if DEBUG
Medium	Unnecessary ATS exception	Removed
Verified	Legal URLs (Terms, Privacy)	Working
Verified	Production payment mode	Enabled

10.2 Pre-Submission Checklist

Item	Status
App icons (all sizes)	Configured

Launch screen	Configured
Info.plist complete	Yes
Privacy policy URL	https://dollor.ai/privacy
Terms of service URL	https://dollor.ai/terms
Support URL	https://dollor.ai
Age rating	4+
Encryption declaration	No encryption (ITSAppUsesNonExemptEncryption = false)
Location usage descriptions	Configured
Camera/Photo usage descriptions	Configured
Microphone usage description	Configured

10.3 Build Configuration

Setting	Debug	Release
Code Signing	Development	Distribution
Optimization	None	Fastest
Debug Info	DWARF with dSYM	DWARF with dSYM
Strip Debug Symbols	No	Yes
isDummyPaymentMode	true	false

Appendix A: Key File Locations

iOS Customer App

```
eatfaircustomer/
├── ViewModels/AuthViewModel.swift      # Authentication logic
├── ViewModels/HomeViewModel.swift      # Home screen data
├── ViewModels/CartViewModel.swift      # Shopping cart
├── Services/PaymentService.swift       # Stripe integration
├── Info.plist                          # App configuration
└── eatfaircustomer.entitlements        # App capabilities
```

Backend

```
dindin/backend/
├── main_new.py                         # FastAPI application
├── models.py                           # SQLAlchemy models
├── order_flow.py                       # Order processing
├── rideshare.py                        # Driver matching
├── addresses.py                        # Address validation
└── requirements.txt                    # Python dependencies
```

Shared Code

```
eatfair-ios-shared/
├── Sources/EatFairShared/
|   ├── Services/P2PAPIService.swift    # API client
|   ├── Models/                         # Shared models
|   └── Config/AppConfig.swift          # Configuration
```

Appendix B: Contact & Support

Platform: Dollor.ai **Support Email:** support@dollor.ai **Legal:** https://dollor.ai/terms, https://dollor.ai/privacy **API Base URL:** https://api.dollor.ai

---

Report generated: December 9, 2025 *Version: 1.0*